QueryGenie — Review-III demo script

Every question below was asked through the app on 2 October (CodeS-1B, **Balanced**, Academic database) and checked against a hand-written correct query. CodeS gives the same answer every time for the same question, so these will behave the same on stage. Re-check after any change: `python research/demo_check.py` (about 1 minute; results in `results/demo_check.md`).

Before you start (5 minutes before)

1. Laptop on charge. No internet needed — everything runs locally.
2. Open QueryGenie (`python desktop/main.py`, or the built app).
3. The model under the question box says **CodeS-1B (Spider)**. If not: click it → CodeS-1B (Spider).
4. Settings (gear, bottom left) → General → **Balanced — recommended, the tested setting**.
5. Left sidebar: **Academic (students & results)** → **New chat**.
6. Ask question 1 once before the panel arrives: the first answer also loads the model, so it is the slowest (9 s in rehearsal, model loading included); the others took 1–5 s.

The demo (about 5 minutes)

#	Type exactly	What appears	Say
1	`List the subjects and the average marks for each subject.`	**4 of 4 queries agree** (green). CN 55.5, DBMS 48.2, OS 48.0. Open the **Chart** tab → bar chart.	"It wrote four different SQL queries, ran all four, and they all agree — so it answers, and picks a chart."
2	`What are the names of the students in the CSE department?`	4 of 4. Asha, Ravi, Sana.	"It found 'CSE' inside the database — the value retriever from the CodeS paper."
3	`Which student scored the highest marks in DBMS?`	4 of 4. Sana. Open **SQL** → a JOIN of two tables.	"It joins the student and result tables by itself."
4	`Show the name and average marks of every student.`	4 of 4. Six students, Sana highest (81.7). Chart tab.	—
5	`Who topped each subject?`	**"I'm not sure which you mean"** — the queries disagree (only 1 of 4 behind its first choice), and it shows the readings to choose from.	"This is our enhancement. Normal text-to-SQL would just give an answer. Ours sees that its own four queries disagree, so it asks instead of guessing. On Spider this makes it right 83.3% of the time when it answers, against 77.8%, with a third fewer wrong answers."
6	`show me all`	**"All of what?"** — it lists the tables to choose from.	"A vague question gets a clarifying question, not a guess."
7	Click **Map** (top left of the chat)	The schema map: student and result tables with their columns and keys. Tap a column → a question about it appears in the box.	"You can see the whole database and ask about any part of it."

Optional, if time: **All 4 queries** tab on any answer shows the four candidate queries and which agree; the **ring** next to the model name shows how much of the model's context the question used.

Keep off stage

These were tested and give a **confident but wrong** answer — don't ask them:

- "Which subject is the hardest?" (answers the subject with the *highest* mark)
- "Which department is the best?" (answers by number of students)
- "Which students failed more than two subjects?…" (2 of 4 agree; wrong rows, shown with a warning)
- Anything on **Car makers & models** (Spider's car data stores numbers as text)
- Follow-up questions ("which of them…") — CodeS answers every question on its own.

If something unexpected happens: "that's exactly why it shows how many of its queries agree — a low-agreement answer is flagged, and under the Strict setting it would decline."

If they ask about the numbers

"Did you reproduce the paper?" Yes. 77.8% Execution Accuracy on Spider dev against 77.9% in the paper (0.1 points), with the paper's own official evaluator. On Spider-Realistic, 69.5% against the paper's 70.9% (1.4 points). The department's tolerance is ±5–10%; both are well inside.

"Why not exactly the paper's number?" Different hardware and libraries: a free Colab T4 and an Apple M4 instead of the authors' 8× A800 GPUs, newer PyTorch/transformers, bf16 on the Mac. We measured the hardware effect: the same 508 questions scored 72.8% on the Mac and 73.2% on Colab. Every run, including failed ones, is in `results/reproduction_log.csv`.

"Did you improve on the paper?" Not overall accuracy — that stays 77.8%. We made it trustworthy: by declining the ~10% of questions where its four queries disagree, it is right on 83.3% of the questions it answers, and wrong answers drop from 230 to 156. Paired t-test $p = 7.7\times10^{-10}$, Wilcoxon $p = 9.8\times10^{-4}$.

"Why not just use majority vote to pick the answer?" We tested it: majority vote *lowers* accuracy to 75.7%. Agreement is a good signal for *whether* to trust an answer, not for *which* answer to give.

"How is this different from ChatGPT?" ChatGPT answers everything with the same confidence and needs your schema sent to a third party. QueryGenie runs locally on open weights and says when it isn't sure.

"What about self-correction (it's in the title)?" It's in the app: a failed query goes back to the model with the error. We measured it honestly: on Spider it fixes few queries (77.76% → 77.85%), because CodeS was never trained to read error messages — which is why abstention became the graded enhancement.

"What about real-world databases (BIRD)?" Started, stopped after 18 of 500 questions because a full run takes over four hours on a laptop; it's future work, with the setup ready (`research/bird_eval.py`).